

Introduction to tangler

Arafat Rahman, PhD

Welcome to tangler

tangler is an R package designed to create visually appealing tanglegrams using the **ggtree** and **ggplot2** ecosystem. This package primarily supports two and three phylogenies to visualize host-symbiont relationships, co-evolution, or simply comparing hierarchical clusterings.

Getting Started

Load the necessary libraries:

```
library(ggplot2)
library(ggtree)
library(dplyr)
library(tangler)
```

Main Functions

1. **common.tanglegram**: Connects tips between two trees using a common categorical metadata column.
2. **simple.tanglegram**: Connects tips between two trees based on a specific matching value within a metadata column.
3. **triple.tanglegram**: Extends the capabilities of **common.tanglegram** to three phylogenies, allowing for the visualization of tripartite associations.

Toy Example

Here's a quick example to generate random trees and plot tanglegrams. First, we'll create three random phylogenetic trees using the **ape** package and assign identical tip labels to each of them to simulate shared taxa.

```
library(ape)

# Create 3 simple trees
set.seed(42)
t1 <- rtree(10)
t2 <- rtree(10)
t3 <- rtree(10)

t1$tip.label <- paste0("t", 1:10)
t2$tip.label <- paste0("t", 1:10)
t3$tip.label <- paste0("t", 1:10)
```

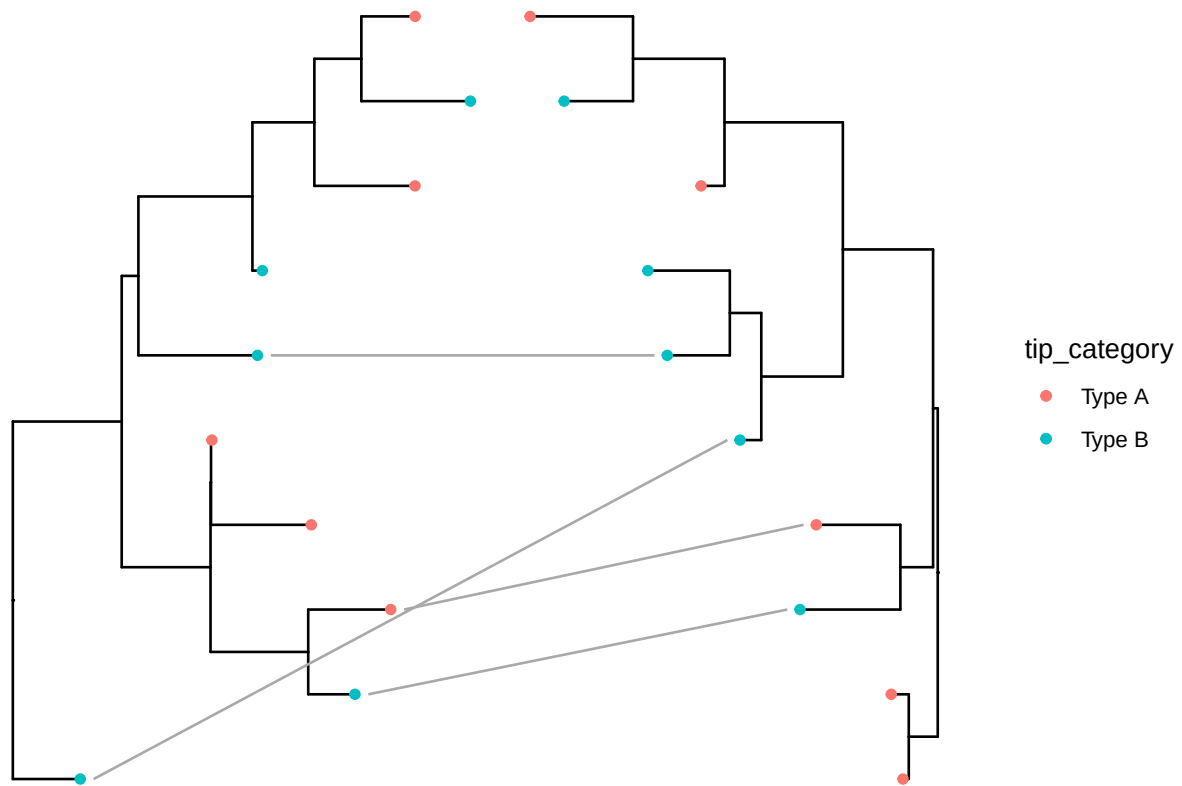
Next, we create a mock metadata data frame containing the connecting information (e.g., categories mapping the tips) and some alternative dummy traits like `tip_category` and `location`. We then attach this metadata to our trees using `ggtree`'s `%<+%` operator.

```
# Create metadata
meta <- data.frame(
  label = paste0("t", 1:10),
  category = sample(c("Group 1", "Group 2", "Group 3"), 10, replace = TRUE),
  tip_category = sample(c("Type A", "Type B"), 10, replace = TRUE),
  location = sample(c("North", "South", "East", "West"), 10, replace = TRUE)
)

# Convert to ggtree objects and join metadata
tree1 <- ggtree(t1) %<+% meta
tree2 <- ggtree(t2) %<+% meta
tree3 <- ggtree(t3) %<+% meta
```

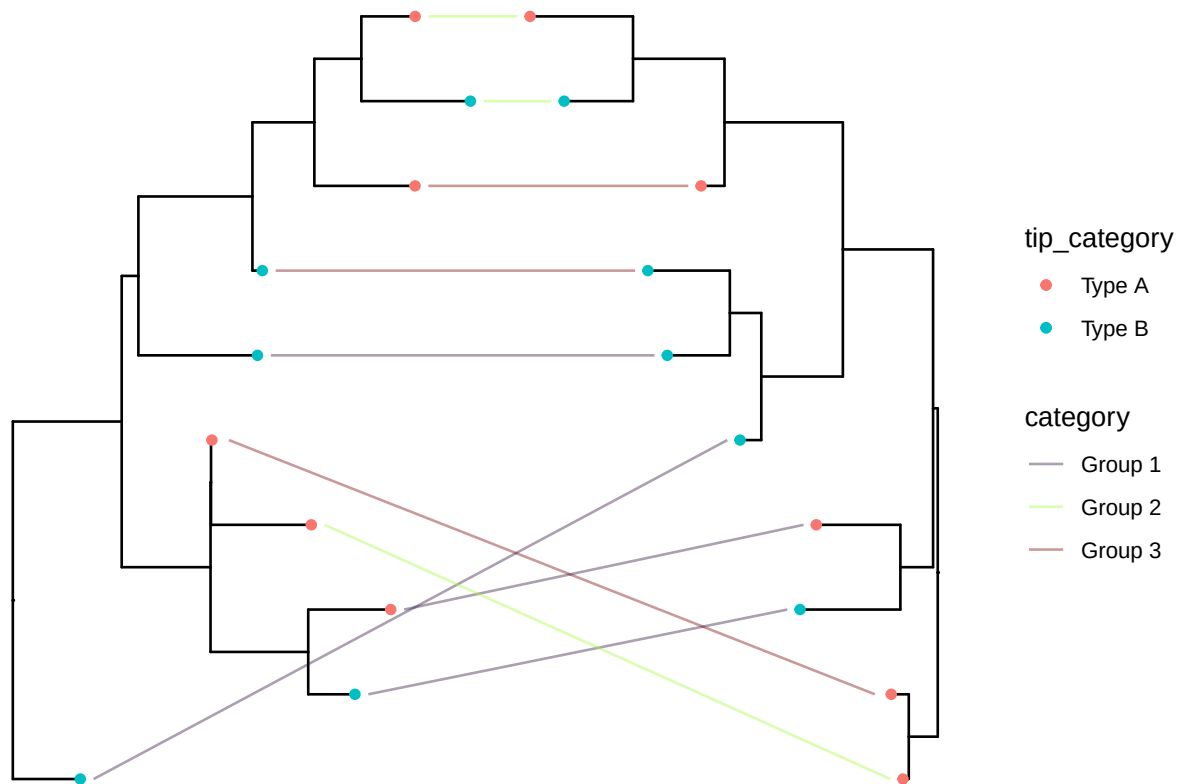
Simple Tanglegram If you only want to plot lines between tips that share a specific value (for example, only showing connections for “Group 1”), use `simple.tanglegram`. Here, we also color the tip points according to the `tip_category` column.

```
# 1. Simple Tanglegram (Filters for "Group 1", colors tips by tip_category)
simple.tanglegram(
  tree1, tree2,
  column = category,
  value = "Group 1",
  tip_column = tip_category
)
```



Common Tanglegram To view links for all tips colored by their overarching category, use `common.tanglegram`.

```
# 2. Common Tanglegram
common.tanglegram(
  tree1, tree2,
  column = "category",
  tip_column = "tip_category",
  t2_pad = 0.5
)
```

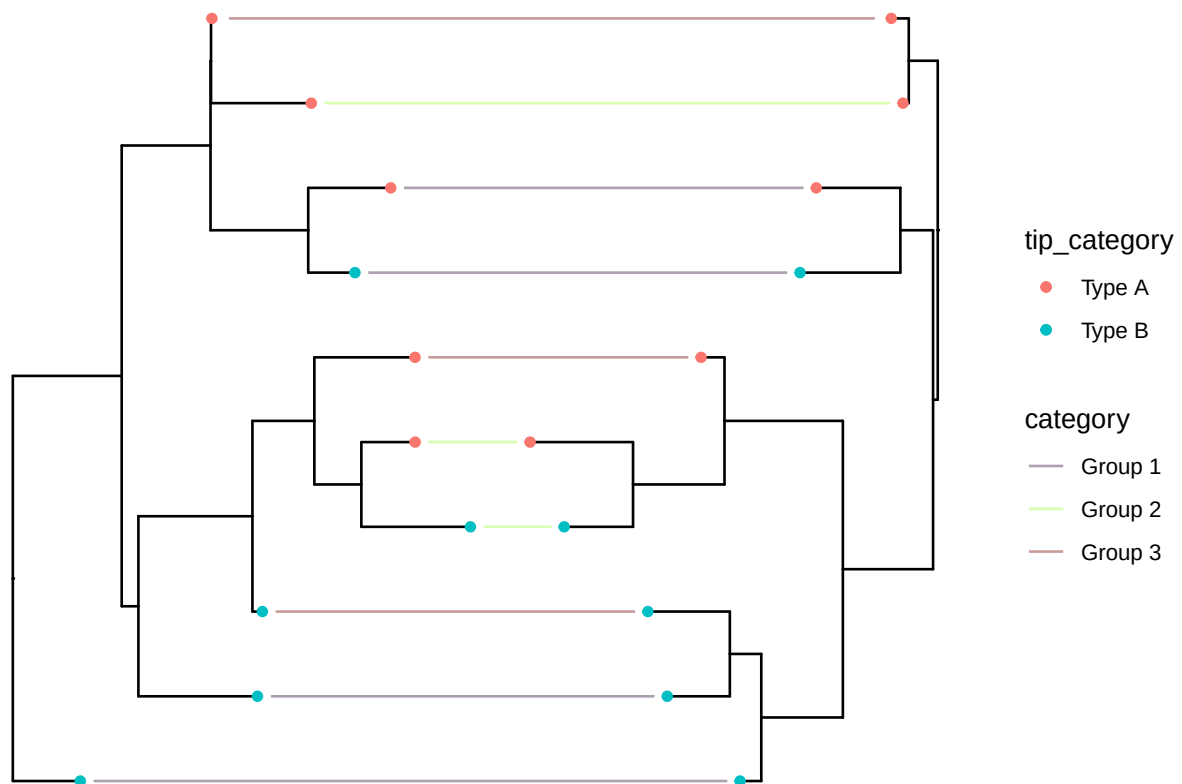


Untangling with `pre.rotate` Often, drawing tanglegrams directly with default trees results in many crossed lines. You can use the `pre.rotate` function to automatically reorder the branches of the trees to minimize these line crossings before plotting.

```
# 2b. Pre-rotate the trees to minimize crossings
rotated_trees <- pre.rotate(t1, t2)
#> Rotating nodes to optimize matching...
#> Done.
rot_t1 <- rotated_trees[[1]]
rot_t2 <- rotated_trees[[2]]

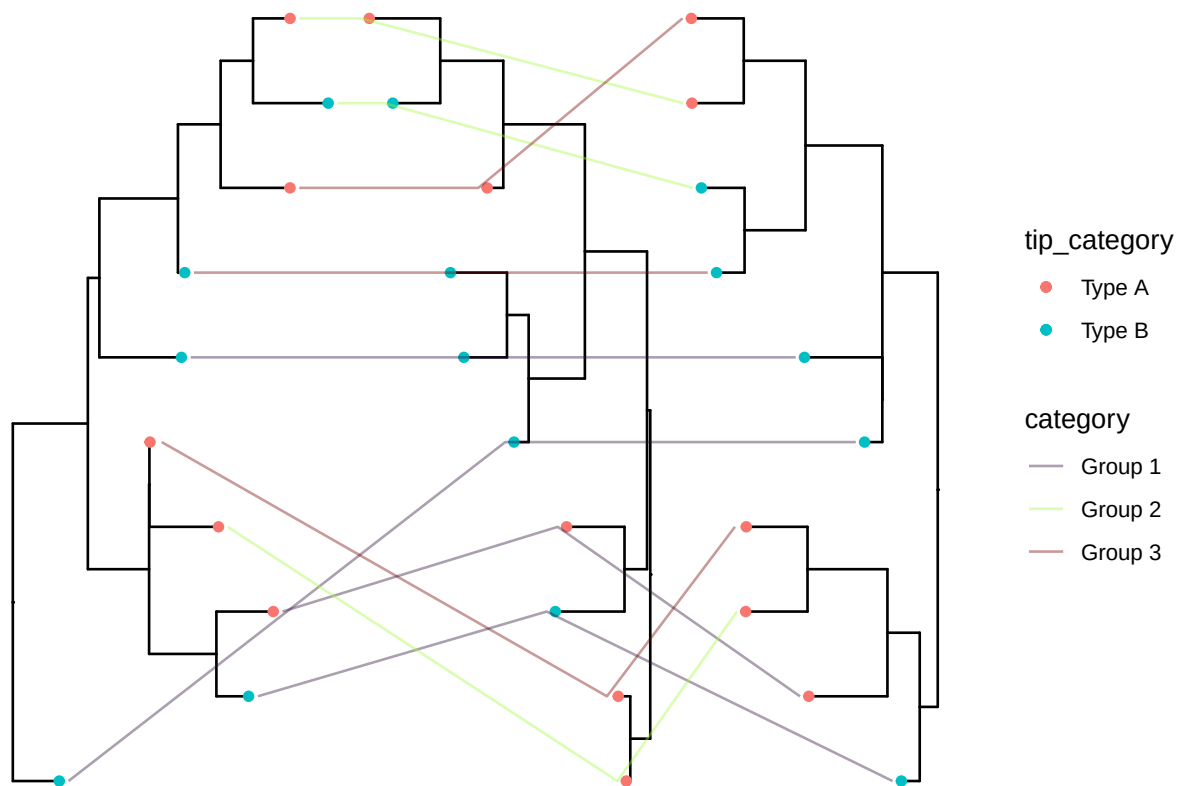
# Convert to ggtree objects and join metadata
# Note: ladderize = FALSE is required so ggtree respects our new rotation
rot_tree1 <- ggtree(rot_t1, ladderize = FALSE) %<+% meta
rot_tree2 <- ggtree(rot_t2, ladderize = FALSE) %<+% meta

# Draw the Common Tanglegram with rotated trees
common.tanglegram(
  rot_tree1, rot_tree2,
  column = "category",
  tip_column = "tip_category",
  t2_pad = 0.5
)
```



Triple Tanglegram For visualizing tripartite relations spanning three phylogenies, use `triple.tanglegram`.

```
# 3. Triple Tanglegram
triple.tanglegram(
  tree1, tree2, tree3,
  column = "category",
  tip_column = "tip_category",
  t2_pad = 0.5,
  t3_pad = 0.5
)
```



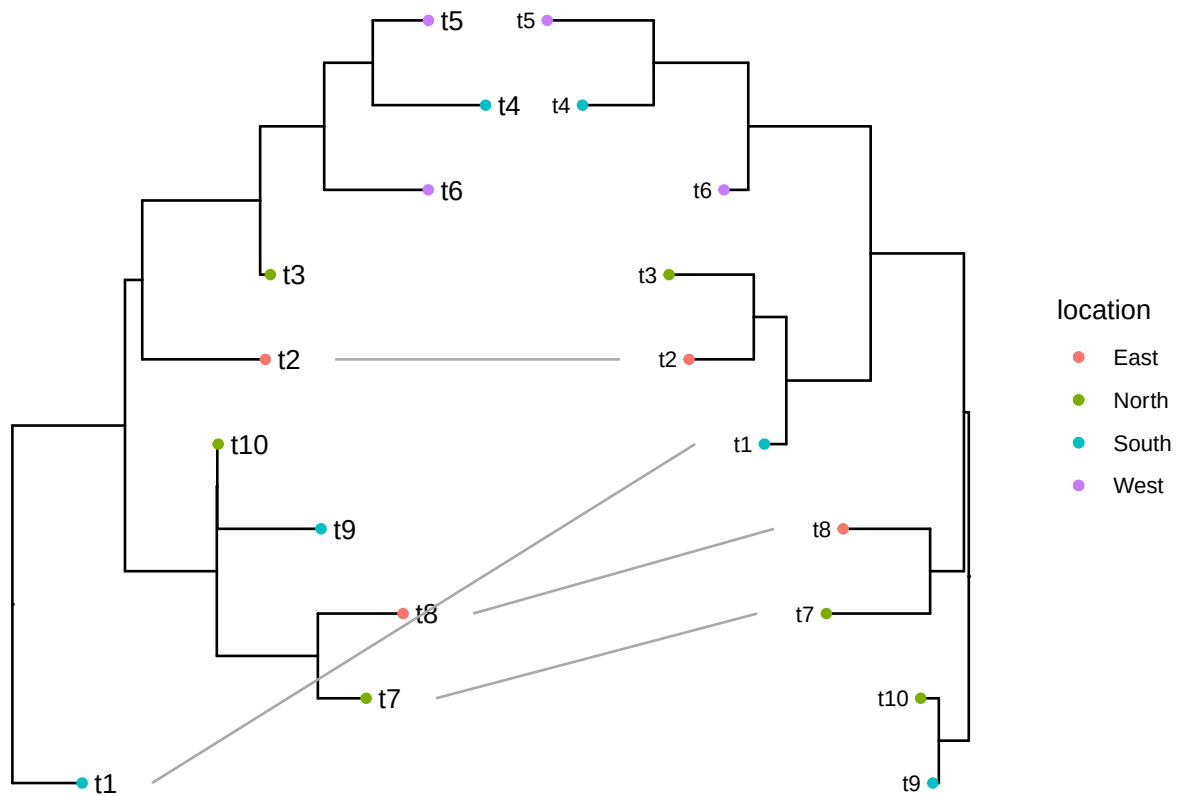
Examples with alternative tip column and tip labels

You can easily swap out the column used for the tip point colors, and optionally enable tip labels on your trees.

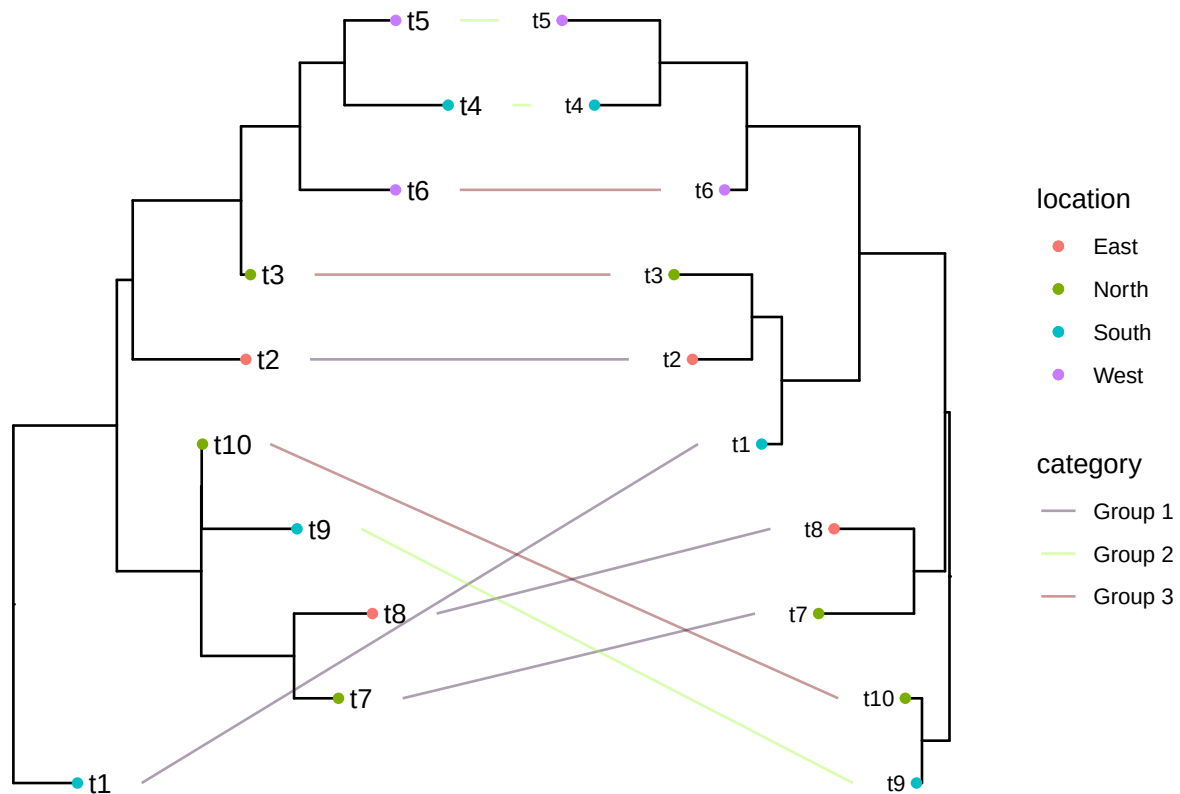
Note that `tree1` needs `geom_tiplab()` manually added, while `tree2` has tip labels appended automatically if you use `tiplab = TRUE` in the two-tree tanglegram functions.

```
# Add labels to tree 1 manually, use tiplab=TRUE for tree 2
tree1_labeled <- tree1 + geom_tiplab(aes(x = x + 0.1))

# 4. Simple Tanglegram with alternative tip column ('location') and tip labels
simple.tanglegram(
  tree1_labeled, tree2,
  column = category,
  value = "Group 1",
  tip_column = location,
  tiplab = TRUE,
  lab_pad = 0.5,
  t2_tiplab_pad = 0.1
)
```



```
# 5. Common Tanglegram with alternative tip column and tip labels
common.tanglegram(
  tree1_labeled, tree2,
  column = "category",
  tip_column = "location",
  tiplab = TRUE,
  t2_pad = 1,
  lab_pad = 0.5,
  t2_tiplab_pad = 0.1
)
```



For the triple tanglegram, since it doesn't natively wrap the `tiplab` toggle, you can just manually add the labels directly to all your trees.

```
# 6. Triple Tanglegram with alternative tip column and tip labels
# Since triple.tanglegram does not have a tiplab argument, we manually
# add labels to tree 2 and tree 3
tree2_labeled <- tree2 + geom_tiplab(aes(x = x - 0.1), hjust = 1)
tree3_labeled <- tree3 + geom_tiplab(aes(x = x - 0.1), hjust = 1)

triple.tanglegram(
  tree1_labeled, tree2_labeled, tree3_labeled,
  column = "category",
  tip_column = "location",
  t2_pad = 1,
  t3_pad = 1,
  lab_pad = 0.5
)
```

